Bagging and Random Forests

Turn unstable trees into a stable ensemble

Ensemble Methods · Foundations of Machine Learning

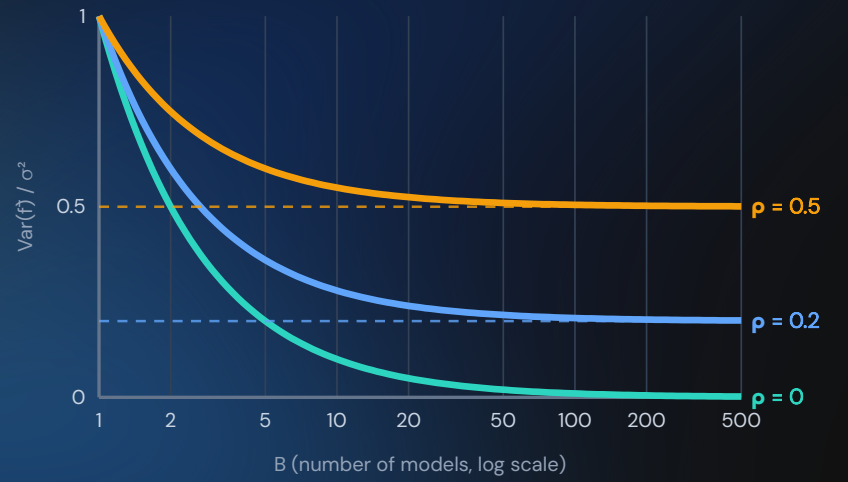
Where We Left Off

$$\mathrm{Var}(\bar{f}) = \underbrace{\rho\sigma^2}_{\text{floor}} + \underbrace{\frac{1-\rho}{B}\sigma^2}_{\to 0 \text{ as } B \text{ grows}}$$

σ² — How much one member's prediction moves when the training data changes.

B — More members shrink the second term. Cheap, but it only reaches the floor.

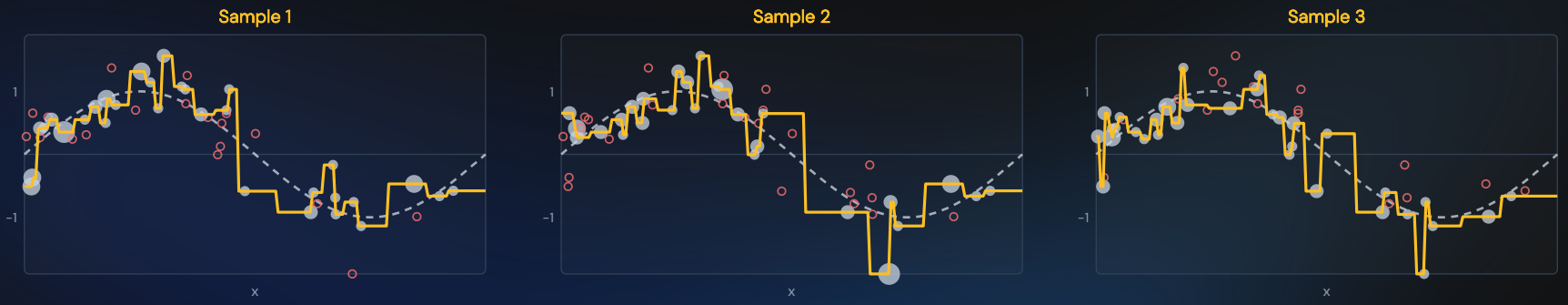
ρ — Lower correlation lowers the floor itself.



Today's question: from one dataset, how do we build many members with small ρ?

One Tree, Three Training Sets

Same 50 points, resampled three times. Dot size = how many times a row was drawn. Red rings = rows left out. Amber = a fully grown tree fit to that sample.



Low bias — Each tree follows the dashed truth on average, wherever there is data.

High variance — Each tree chases its own sample's noise. The spikes differ from panel to panel.

Bootstrap Sampling

Draw n rows from n rows, with replacement. Each draw picks any row with probability 1/n.



Same size

Every sample has n rows, so each tree trains on a dataset as large as the original.

Repeats and gaps

Dashed tiles are repeats. Rows never drawn are out-of-bag for that sample.

A stand-in for new data

Each resample imitates a fresh training set drawn from the population.

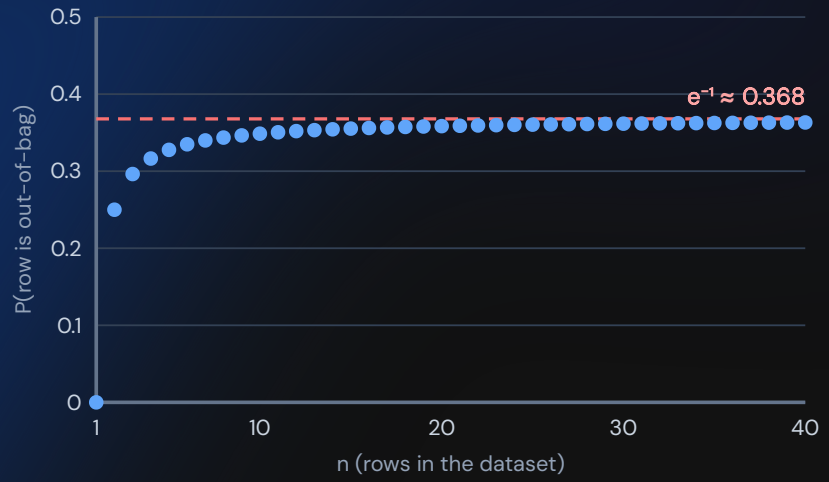
Why About 36.8% Are Left Out

One draw misses row i — with probability 1 - 1/n.

All n draws miss it — The draws are independent, so multiply.

$$P(\text{out-of-bag}) = \left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368$$

Worked example: with n = 1,000 rows, each sample still has 1,000 draws, but about 368 distinct rows never appear. The other ~632 rows fill all 1,000 draws, some of them more than once.

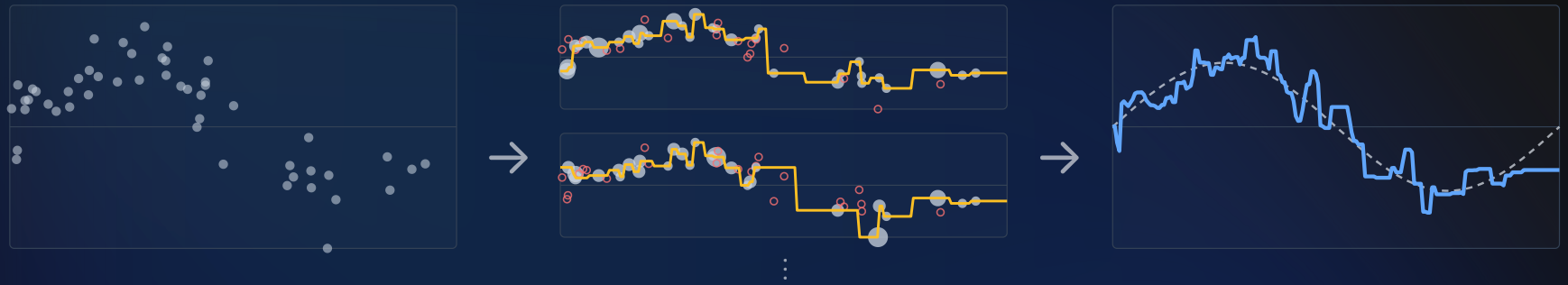


Bagging = Bootstrap + Aggregate

1 · One dataset

2 · B samples, B deep trees

3 · Average (or vote)



$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x)$$

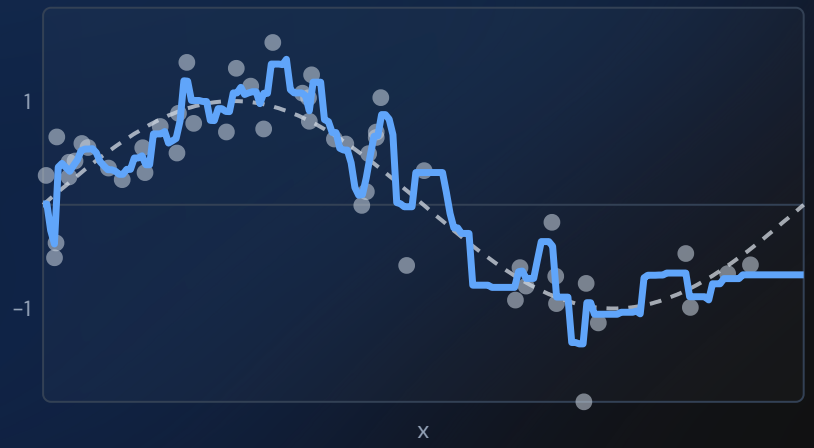
Classification: majority vote of the B trees, or average their predicted probabilities and take the largest.

Averaging in Action

25 bootstrapped trees



Their average



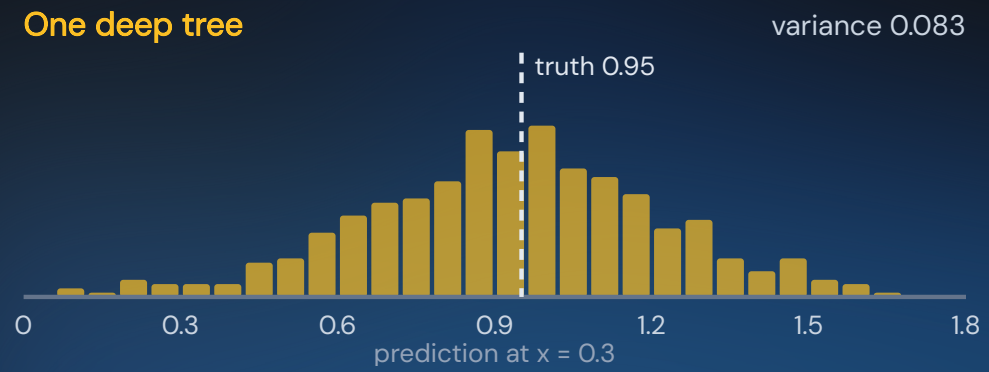
Individually — Every tree is jagged, and each in its own places.

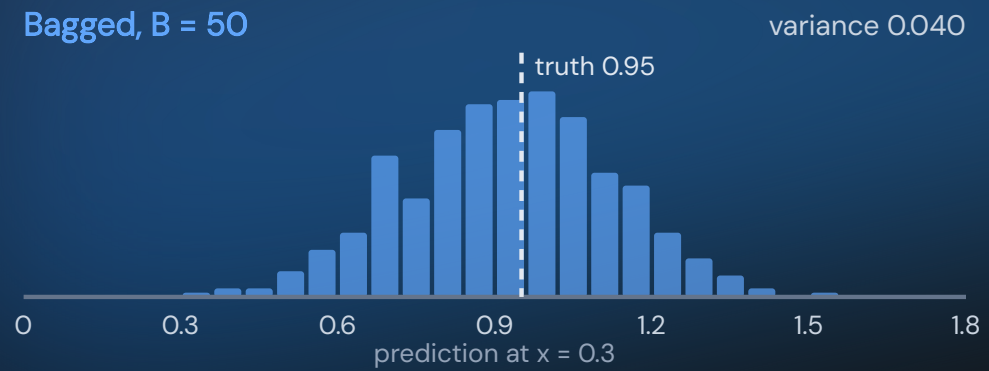
Together — The spikes point in different directions and largely cancel. The shape they share remains.

Averaging keeps what the trees agree on (the signal) and cancels what they disagree on (the noise).

One Input, 400 Training Sets

Draw a fresh 50-point dataset 400 times. Each time, record the prediction at x = 0.3 from one deep tree and from a bagged ensemble of 50 trees.





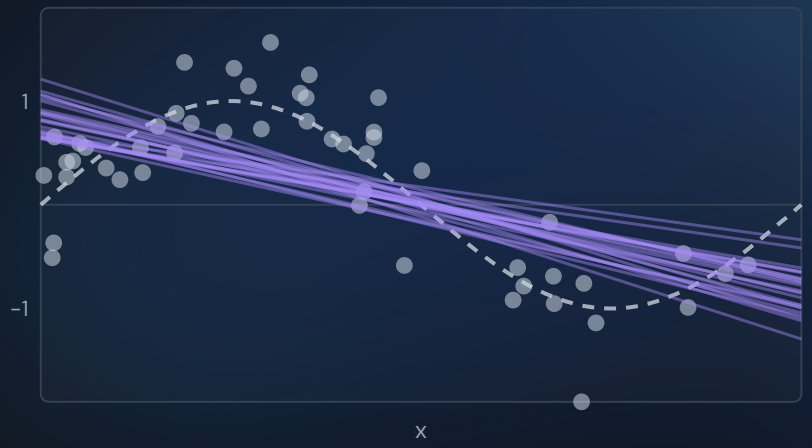
Same center — Mean prediction 0.928 vs 0.924. Truth is 0.951. Bias is essentially unchanged.

Half the spread — Variance 0.083 → 0.040.

Why not 50× less? — The implied ρ between bagged trees is 0.41. The floor $\rho\sigma^2$ = 0.41 × 0.095 ≈ 0.039 is where we landed.

Why Trees? Bagging Needs Variance to Remove

25 bootstrapped linear fits



25 bootstrapped deep trees



Linear regression: stable but biased — The lines barely move. Averaging them gives the same wrong line. Bagging has nothing to cancel.

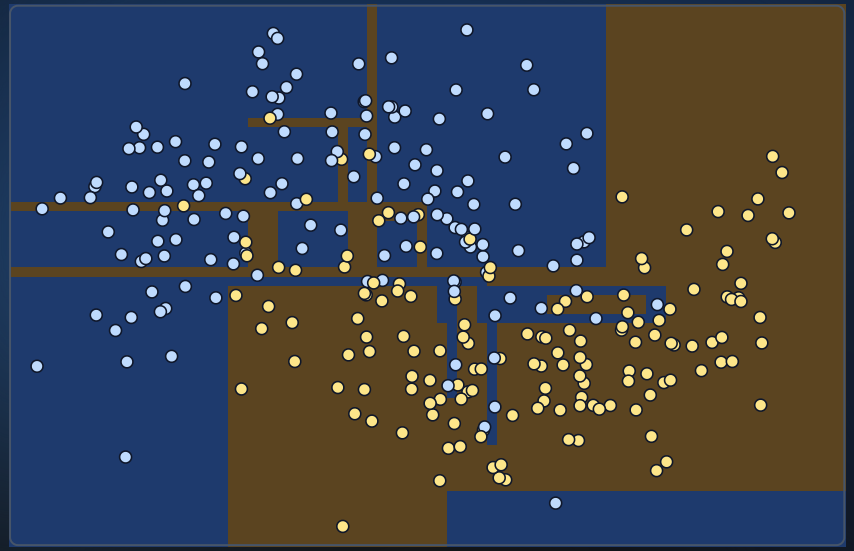
Deep tree: accurate but unstable — Right on average, wrong in different places each time. Exactly what averaging fixes.

Bag a learner when it is **low bias** and **high variance**. Deep trees are the textbook case.

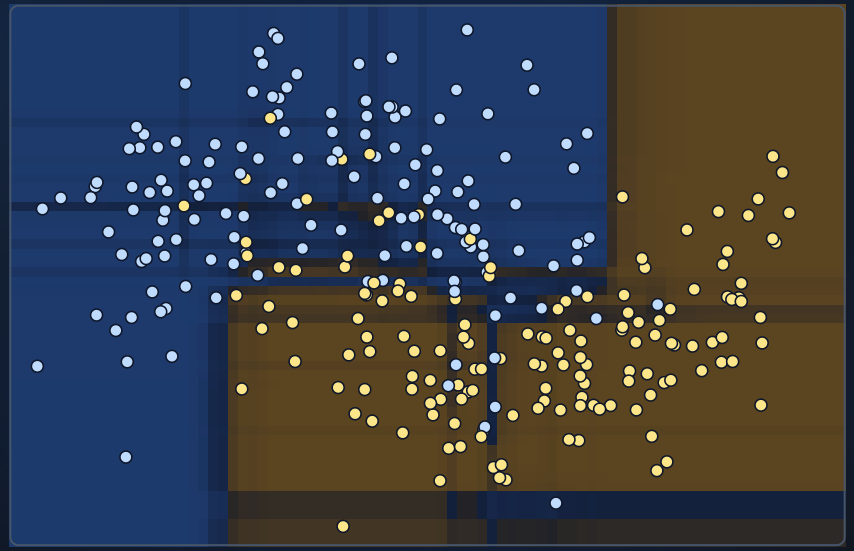
Bagging Classifiers: Smoother Boundaries

Two moons, 300 training points. Shading = predicted probability of the amber class. Accuracy on 5,000 fresh points.

One deep tree — test accuracy 83.4%



Bagged, 300 trees — test accuracy 88.3%



One tree — Every region is fully confident. Small islands around single points.

Bagged — Islands mostly vote away. Pale bands mark where trees disagree.

The Correlation Problem

California housing: predict median house value from 8 features, 2,000 training rows. Which feature does each bagged tree split on first?



300 of 300 on MedInc — Median income is so predictive that every bootstrap sample picks it first.

Same top, similar trees — The first split shapes everything below it, so the trees share much of their structure.

Shared structure, shared mistakes — ρ is higher than it needs to be, and so is the floor, however many trees we add.

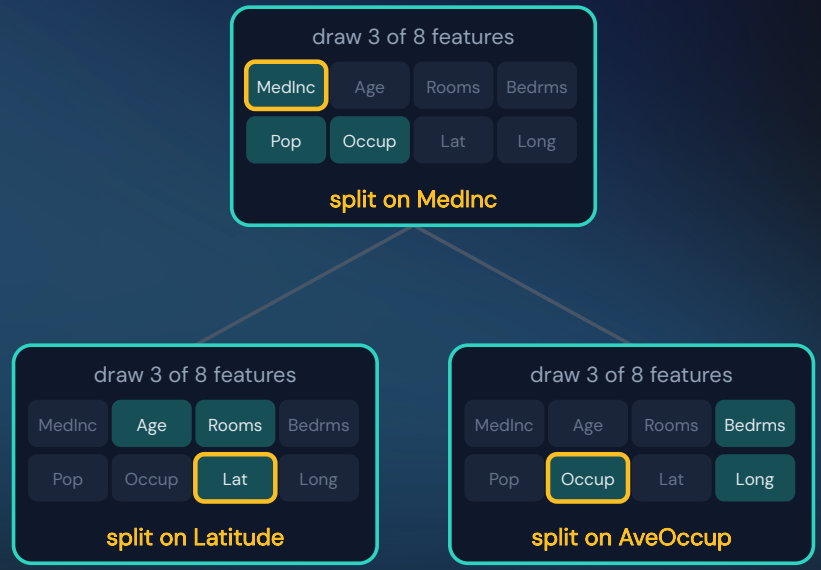
Random Forests: Random Features per Split

At every node — Draw m of the d features at random. Search for the best split among those m only.

The dominant feature is sometimes unavailable — Then the tree must split on something else. Different roots, different trees.

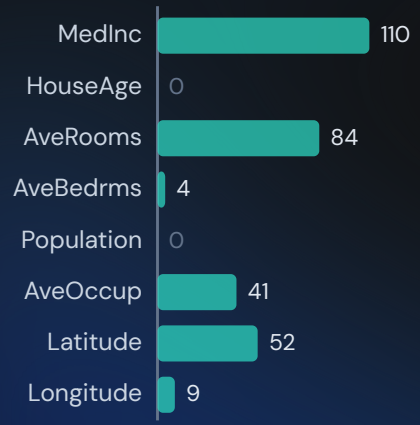
Random forest = bagging + this — With m = d it is exactly bagging.

Common starting points: `m = √d` for classification, `m ≈ d/3` for regression. Tune it.



Decorrelation, Measured

Random forest (m = 3 per split)



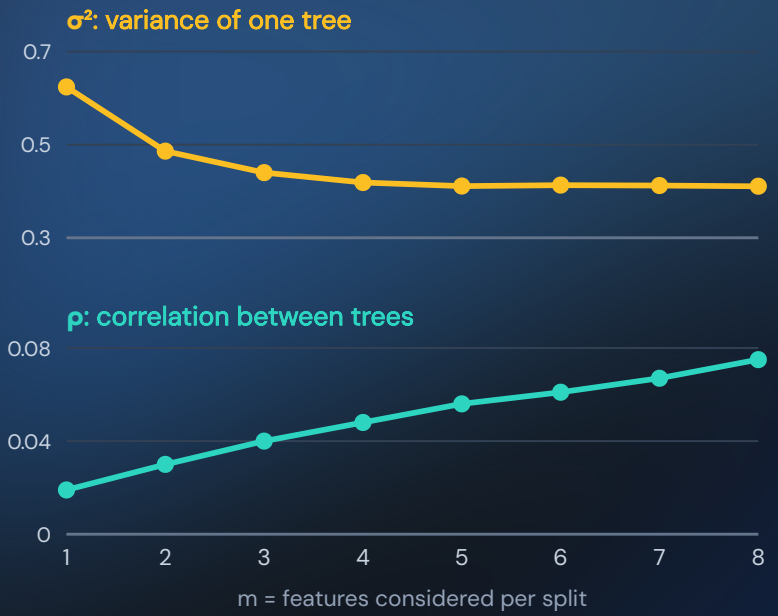
MedInc is in the root's draw with probability 3/8. Expected 112.5 of 300 trees; observed 110.

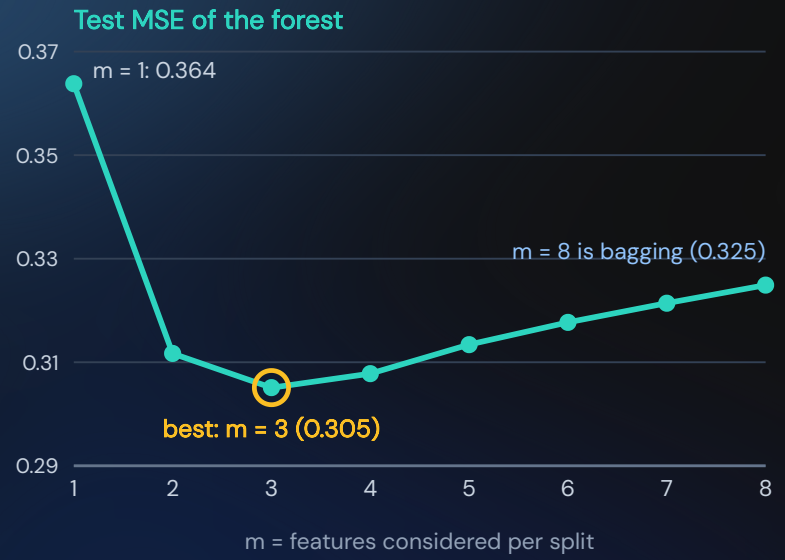
Measured over 20 training sets

	σ^2 (one tree)	ρ	floor $\rho\sigma^2$
Bagging, m = 8	0.411	0.075	0.031
Forest, m = 3	0.440	0.040	0.018

The trade: each tree got a little worse (σ^2 up 7%), but the trees got much less alike (ρ down 47%). The floor drops by about 43%.

Choosing m: Strength vs. Diversity



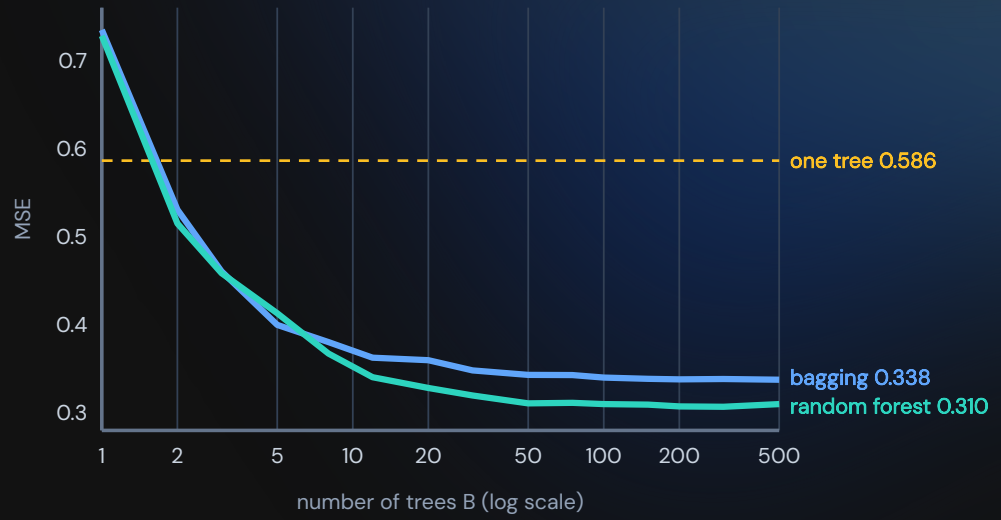


Small m — Very diverse trees, but each one is weak.

Large m — Strong trees, but they all look alike.

In between — The best m depends on the data. Tune it with OOB or CV.

How Many Trees?



Steep, then flat — The $(1 - \rho)\sigma^2/B$ term vanishes fast. Most of the gain comes in the first ~50 trees.

Different plateaus — Bagging levels off near 0.338, the forest near 0.31. The gap comes mostly from ρ, not B.

More trees never hurt accuracy — Only time and memory. Unlike boosting rounds, B is not a knob that overfits.

Out-of-Bag Evaluation



row 2 is out-of-bag for 4 of 10 trees → T1, T2, T3, T9 vote

For each training row — Predict it using only the trees that never saw it.

Score all rows that way — The result is an estimate of test error, with no held-out split.

How many voters? — About 36.8% of B. With 200 trees, ~74 per row.

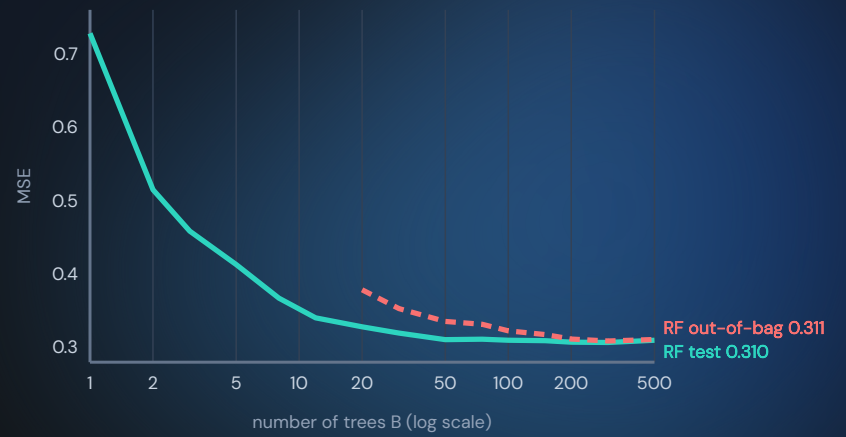
Out-of-Bag in Practice

```python
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=500, max_features=3,
    oob_score=True, n_jobs=-1, random_state=0,
)
rf.fit(X_train, y_train)

print(rf.oob_score_)  # R² of the OOB predictions
oob_mse = ((rf.oob_prediction_ - y_train) ** 2).mean()
```

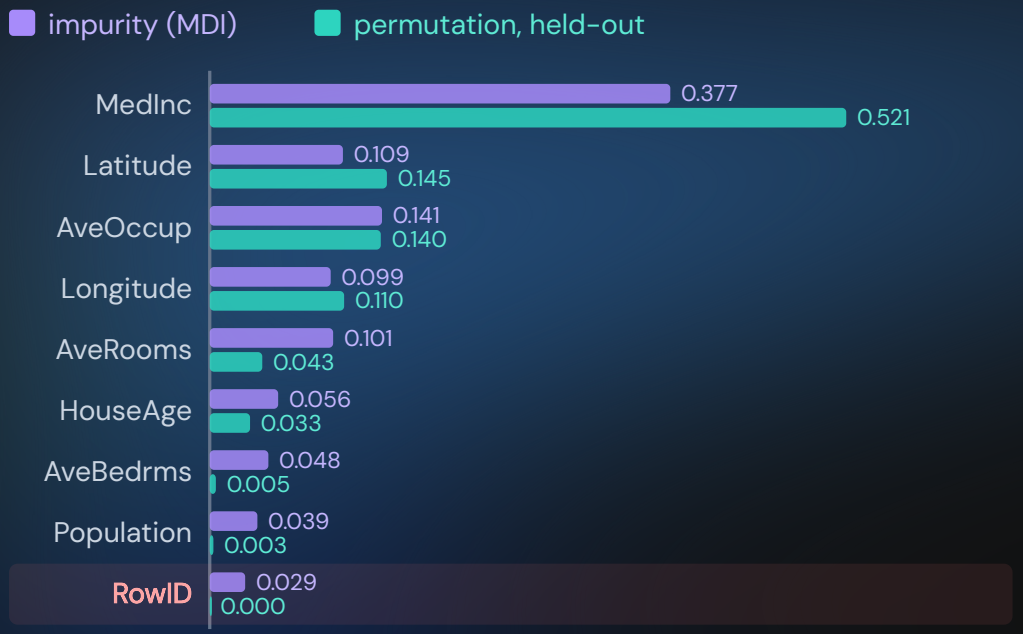
Use it to choose `max_features` or to check that B is large enough, without spending data on a validation split.



At 500 trees: out-of-bag MSE 0.311, test MSE 0.310. Out-of-bag is a little high for small B, when each row has few voters.

Feature Importance: Useful, Not Causal

Same forest, plus a RowID column: a random row number with no relationship to price.



Impurity (MDI) — Total split improvement per feature, on the training data. Fast, but RowID gets 2.9%.

Permutation — Shuffle one column on held-out data and measure how much error rises. RowID gets 0.

Neither is causal — Both describe what this model relies on. Correlated features split the credit.

Practical Defaults

Hyperparameter	Controls	Starting point
`n_estimators`	Number of trees B	A few hundred. Increase until OOB error stops falling.
`max_features`	m, decorrelation	Classifier: `"sqrt"` (default). Regressor: an integer near d/3; the default `1.0` is plain bagging.
`min_samples_leaf`	How deep each tree grows	Fully grown (defaults). Raise `min_samples_leaf` for very noisy data.
`oob_score`	Free validation estimate	`True` while tuning.
`n_jobs`	Parallel training	`-1` , all cores.

A strong first baseline

Works on tabular data with little preprocessing: no scaling, and it tolerates irrelevant features.

Forgiving to tune

Tune m first. B only needs to be large enough.

Bagging and Random Forests

Bootstrap

Resample rows to get many training sets from one. ~36.8% of rows are left out of each.

Bagging

Average deep trees. Variance falls to the floor $\rho\sigma^2$; bias stays put.

Random forest

Sample m features per split. Lower ρ, lower floor. Tune m; out-of-bag error comes free.

Next: boosting trains weak learners sequentially, each correcting what remains wrong.